Integrantes: Alison Betancourt, Fernando Huilca, Mateo Simbaña

Curso: GR3SW

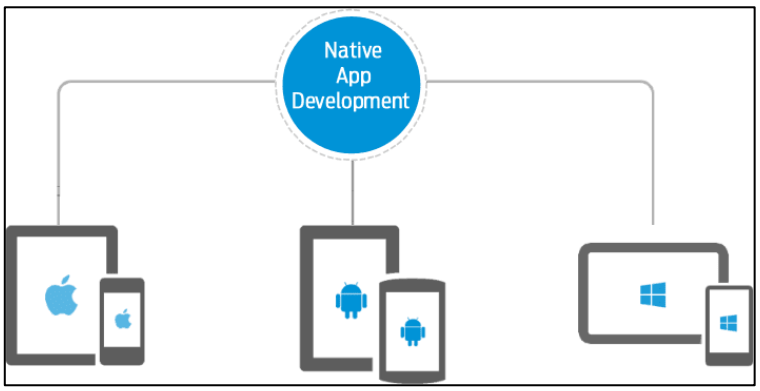
Trabajo en clase 01

Parte 1: El Viaje Arquitectónico del Desarrollo Móvil

1. Desarrollo nativo puro

La historia no comenzó como la conocemos. Cuando Steve Jobs presentó el iPhone en 2007, su visión era radicalmente distinta: no quería que nadie instalara software externo. Su frase fue: "La solución completa son las Web Apps 2.0". Jobs creía que el navegador Safari era suficiente.

Sin embargo, los desarrolladores odiaron la idea. Las aplicaciones web de 2007 eran lentas, no tenían acceso a la cámara y se sentían "ajenas". La presión fue tal que Apple tuvo que ceder, y en marzo de 2008 lanzó el SDK de iPhone. Fue el nacimiento del Nativo Puro: código escrito exclusivamente para un hardware específico.



- ***Dos Mundos, Cero Puentes***

En esta era si eras una startup se tenía que ir por dos caminos, no existía el “compartir códigos”.

Por un lado, estaba el artesano de iOS porque programar para iPhone era como hablar un dialecto antiguo, se usaba Objective-C, un lenguaje que mezclaba la potencia bruta de C con la elegancia de Smalltalk. Para evitar que las apps se cerraran por falta de memoria, Apple perfeccionó el Automatic Reference Counting (ARC), que a diferencia de otros sistemas no hay un proceso como si se limpiara la basura, sino el compilador inserta instrucciones para borrar de manera automática, es como si en un restaurante el mesero retirara el plato en el segundo exacto en el que se termina de comer, sin que nadie se lo pida. Todo se procesaba con el compilador LLVM, que transformaba el código en instrucciones que el chip de Apple entendía de forma nativa.

Por otro lado, estaba el ingeniero de Android porque Google había tomado un camino distinto, como había miles de teléfonos diferentes, no podían compilar para uno solo, por

ello usaron Java y una máquina virtual llamada Dalvik (luego evolucionada a ART), el código no iba directo al procesador, se transformaba en un "Bytecode" que la máquina virtual interpretaba, esto permitía que la App funcionara en un Samsung, un LG o un Motorola sin cambiar una línea de código, la desventaja era el rendimiento, las apps de Android de esa época a veces se sentían pesadas comparadas con las de iOS.

- ***Por qué el Nativo es inigualable en cuestión de rendimiento***

Lo que hace al Nativo Puro algo único es la Latencia de Interfaz, el hilo principal (el que dibuja la pantalla) es sagrado, el SO le da prioridad absoluta, si el procesador esta ocupado calculando algo difícil, el nativo pausa eso asegurándose de que el dedo mueva en la pantalla sin lag. El código nativo, habla directo con los sensores, por ejemplo, si se usa el FaceID, la respuesta es inmediata porque no hay capas de traducción en el medio, es una conexión eléctrica casi directa entre el código y el sensor.

- ***Qué pasa actualmente***

Tal vez se pensó que con la llegada de Flutter moriría el nativo, pero no fue así, hay en tres casos específicos donde es indiscutible:

  - Si se va a procesar video 4K en tiempo real o hacer cálculos de Realidad Aumentada (AR), se necesita Nativo, frameworks híbridos añaden milisegundos de retraso que arruinan la experiencia.
  - Las pantallas plegables requieren que la arquitectura de la app sea reactiva al hardware, el desarrollo nativo permite usar librerías como Jetpack WindowManager para saber exactamente cuándo la pantalla se está doblando y ajustar la interfaz en milisegundos sin perder el estado del usuario.
  - Las apps que requieren niveles de encriptación militar prefieren nativo porque tienen control total sobre cómo se almacena la información en los chips de seguridad del teléfono.

- ***Tecnologías Actuales***

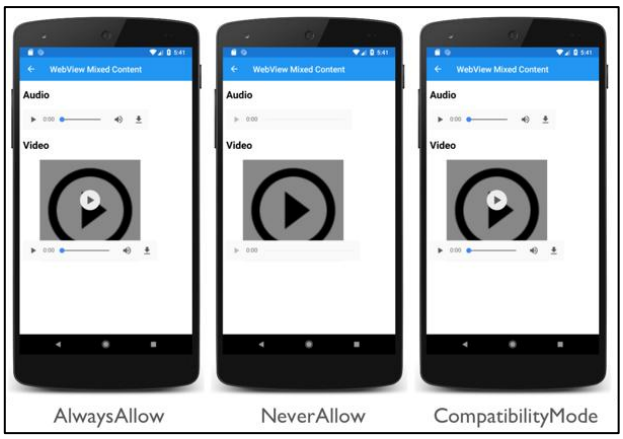
iOS ya no tiene como lenguaje estándar Objective-C, es Swift y su motor gráfico es Metal, Android tiene como lenguaje Kotlin y su motor gráfico es Vulkan y ambos han migrado a interfaces Declarativas (SwiftUI y Jetpack Compose), donde tú le dices a la app qué quieres ver y ella decide *cómo* dibujarlo eficientemente.

El Nativo Puro es la perfección técnica, pero su pecado fue la economía. Era demasiado caro mantener dos ejércitos de programadores. Por eso, el mundo empezó a mirar con deseo hacia algo que todos ya sabían usar: el navegador web.

2. La era del WebView

Alrededor del 2011, los desarrolladores web quisieron conquistar el móvil y surgió la idea de meter un sitio web dentro de una aplicación.

Aquí se usan tecnologías como HTML5, CSS y JavaScript. Funciona creando un “contenerdor” nativo que abre un WebView. Desntro de ese componente se cargaba un archivo index.html local. Todo lo que el usuario veía era una página web sin barra de direcciones.



Su fracaso tuvo tres fuentes:

a. El cuello de botella del DOM: En el desarrollo nativo, los elementos (listas, botones) son objetos que el sistema operativo dibuja eficientemente. En un WebView, cada elemente es un nodo en el DOM, el problema se da cuando se hace scroll en una lista de muchos elementos por ejemplo 100 elementos, el motor del navegador tiene que recalcular el diseño y volver a dibujar constantemente. Esto consume CPU de forma masiva, provocando el famosos jank o lag visual.
b. El retraso de los 300ms: Históricamente, los navegadores móviles esperaban 300ms después de un toque para ver si el usuario iba a hacer un "doble tap" para hacer zoom. Las apps híbridas se sentían "pegajosas". Tocabas un botón y no pasaba nada instantáneamente. Ese pequeño retraso era suficiente para que el cerebro del usuario detectara que algo "no estaba bien".
c. La Interfaz de Usuario: Ionic (en sus versiones 1 y 2) intentó imitar los componentes de iOS y Android usando CSS. El botón se *parecía* al de iOS, pero no tenía la misma física de presión, ni la misma sombra, ni la misma inercia al soltarlo. El usuario sentía una incomodidad subconsciente: la app se veía como un impostor.

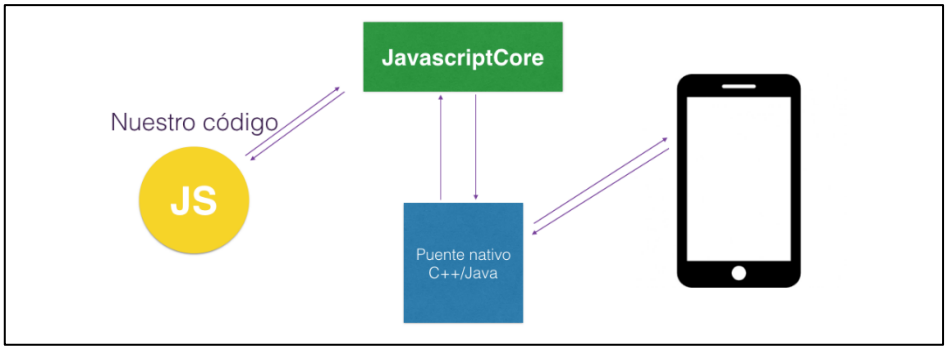
Hoy, esta arquitectura está en declive terminal para aplicaciones de consumo masivo, pero ha mutado en dos vertientes:

a. Capacitor / Ionic Moderno: Han evolucionado mucho. Ya no usan el viejo y lento UIWebView, sino WKWebView, que tiene un motor de renderizado mucho más potente y soporte para aceleración por hardware. Se usan mucho en apps corporativas internas donde el rendimiento extremo no es prioridad.

b. **PWA (Progressive Web Apps):** Es la evolución honesta. Ya no intentan "disfrazarse" de apps nativas en la tienda; son sitios web que aprovechan las capacidades modernas del navegador.

3. **El puente Nativo-JavaScript y otros puentes**

En lugar de renderizar una página web dentro de un WebView, permitó escribir lógica e interfaz en JavaScript/TypeScript y conectarlas con componentes nativos mediante un **bridge**.



Esta familia arquitectónica combina:

- Reutilización de lógica.
- Experiencia cercana al desarrollo web.
- Acceso a componentes del sistema.
- Posibilidad de escribir módulos nativos cuando hace falta.

El enfoque del puente fue una solución elegante al problema de “compartir código sin caer en WebView”, pero sus límites aparecen cuando hay mucha comunicación intensiva entre JavaScript y nativo, puesto que el puente se convierte en cuello de botella.

4. **El motor de renderizado propio**

En vez de apoyarse en widgets nativos de cada plataforma, Flutter dibuja su interfaz mediante su propio framework y motor gráfico.

Esto trae varias ventajas:

- Comportamiento visual muy consistente entre iOS y Android.

- Animaciones suaves.
- Menor dependencia de componentes visuales nativos.
- Una experiencia de desarrollo muy controlada por el mismo framework.

Sin embargo, al no depender de componentes nativos, requiere mayor control del renderizado y puede presentar desafíos al interactuar con funcionalidades específicas del dispositivo.

5. **El futuro: foldables, AR y wearables**

Las nuevas interfaces cambian el problema arquitectónico. Ya no basta con mostrar pantallas estáticas en un rectángulo fijo.

- **Foldables**

En teléfonos plegables, la aplicación debe adaptarse a cambios drásticos de tamaño y orientación sin perder estado. Eso favorece arquitecturas reactivas, con buen manejo de estado, recomposición rápida de interfaz y separación clara entre lógica y presentación. En ese punto, tanto Flutter como React Native pueden adaptarse razonablemente bien si la app fue diseñada de forma reactiva.

- **Wearables**

Los wearables exigen eficiencia energética, interfaces simplificadas y adaptación a APIs muy específicas del dispositivo. Aquí suele reaparecer la ventaja del nativo, porque los relojes y dispositivos pequeños tienen restricciones particulares y APIs del fabricante.

AR/XR

En AR/XR el problema cambia radicalmente, aparecen renderizado 3D en tiempo real, seguimiento espacial, sensores, cámaras, matemáticas de pose y latencia bajísima. Ni React Native ni Flutter fueron diseñados originalmente como frameworks de motor 3D inmersivo; en la práctica dependen de integraciones nativas o motores externos para cubrir esos casos. Flutter reconoce en su propia arquitectura que su motor se encarga de rasterizar y administrar runtime, pero eso no lo convierte en un motor de experiencia XR; React Native, por su lado, se centra en UI y comunicación con nativo, no en renderizado inmersivo de muy baja latencia.

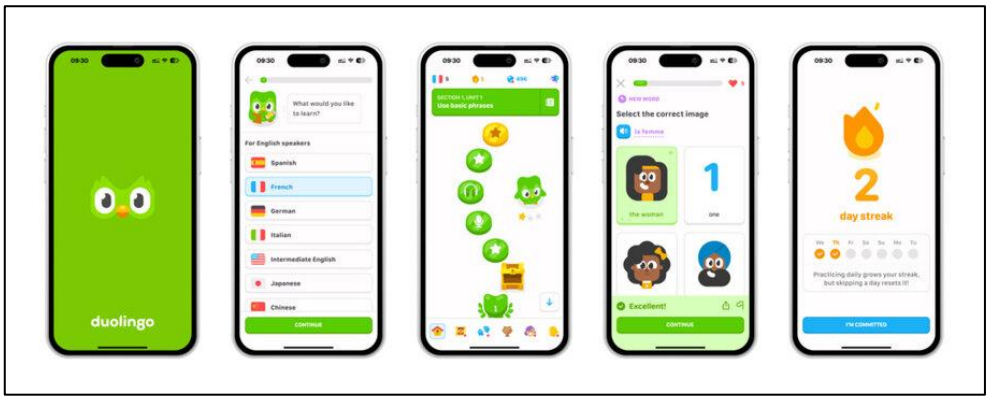
Parte 2: Taller práctico - Análisis crítico de aplicaciones

1. Ingeniería inversa de la decisión

Caso de Estudio A: Duolingo (El camino del Lenguaje Compartido)

Duolingo no es solo una app de idiomas; es un motor de videojuegos de baja intensidad disfrazado de aplicación educativa.

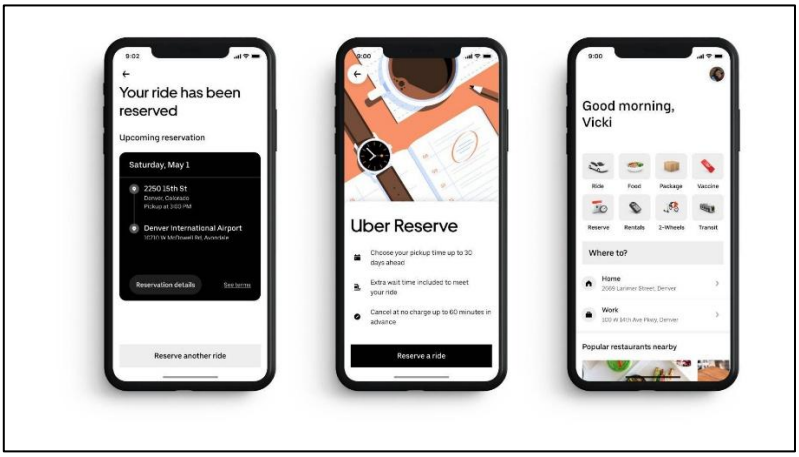
- **¿Por qué Nativo?** Originalmente, Duolingo utilizaba una técnica muy sofisticada: escribían la lógica de las lecciones en **C++** o usaban transpiladores (como **J2ObjC**) para compartir el "cerebro" de la app, pero mantenían la **UI 100% Nativa**.
- **La Razón:** Las animaciones. El búho Duo y las celebraciones tras cada lección requieren una sincronización perfecta y 60 FPS constantes. Intentar hacer esto en un WebView en 2014 habría sido un suicidio comercial. Apostaron por lo nativo porque su producto depende de la "gratificación instantánea" visual.



Caso de Estudio B: Uber (La Orquesta de la Realidad)

Uber es, probablemente, la aplicación nativa más compleja del planeta. No vive en el teléfono; vive en la calle, moviéndose a 60 km/h.

- **¿Por qué Nativo?** Uber necesita acceso quirúrgico al GPS, al acelerómetro y a los servicios de mapas en segundo plano. En sus inicios, la estabilidad de las conexiones y el consumo de batería eran críticos.
- **La Razón:** Crearon su propia arquitectura llamada **RIBs** (Router, Interactor, Builder) para permitir que cientos de ingenieros trabajaran en el mismo código sin romperse los pies unos a otros. Solo lo nativo les permitía tal nivel de control sobre la memoria y los hilos de ejecución (multithreading).



2. **Análisis de rendimiento y arquitectura**

- **Desafío de Uber (Mapas y Tiempo Real):** El mayor desafío es el renderizado del mapa 3D con decenas de vehículos moviéndose simultáneamente. Al ser nativo, Uber puede usar **OpenGL o Metal** directamente. Si usaran un "puente" (Bridge) como el de React Native, cada actualización de la posición de un auto tendría que cruzar del hilo de datos al hilo de UI, generando un retraso (lag) visual que haría que el auto "saltara" en el mapa en lugar de deslizarse.

- **Desafío de Duolingo (Fluidez en Gama Baja):** Duolingo se usa en países en desarrollo con teléfonos muy básicos. Para lograr un scroll fluido en la lista de ligas o amigos, optimizan el reciclaje de vistas nativas. El cuello de botella que evitaron al no usar un Bridge fue la **serialización de JSON**. En un Bridge, cada mensaje de "dibuja esto" debe convertirse a texto y luego a código nativo; Duolingo elimina ese intermediario para que incluso un teléfono de hace 5 años se sienta rápido.

3. **El Desafío de las Nuevas Pantallas (Lentes AR)**

Si el futuro son lentes de Realidad Aumentada:

- **Veredicto: Ambas sufrirían, pero lo Nativo ganaría.**

- **Por qué:** La AR requiere una latencia menor a **20 milisegundos** entre que mueves la cabeza y la imagen se actualiza.

    - **React Native** fallaría porque el "Bridge" introduce demasiada latencia.

    - **Flutter** tendría una oportunidad gracias a su motor de renderizado propio, pero aún tendría que comunicarse con las APIs de los lentes a través de "Platform Channels", lo que añade fricción.

- La arquitectura de **Uber/Duolingo (Nativa)** es la más adaptable porque ya están cerca del hardware y de las librerías matemáticas de bajo nivel (C++ o Swift) necesarias para cálculos de visión artificial.

4. **Veredicto Crítico: ¿Qué elegir hoy?**

Para una app con **mapas 3D interactivos, alta frecuencia de notificaciones y soporte para plegables**, la decisión de ingeniería es:

Elegiría Nativo (Kotlin/Swift).

Fundamentación Técnica:

- **Mapas 3D:** Los SDKs de mapas más avanzados (Mapbox, Google Maps SDK) están optimizados para drivers nativos. Cualquier capa intermedia reduce el rendimiento del sombreado y el zoom.
- **Notificaciones de Alta Frecuencia:** Manejar flujos constantes de datos (sockets) en segundo plano es más estable en el sistema operativo base, evitando que el recolector de basura (Garbage Collector) de un framework cross-platform cierre la app por consumo excesivo de recursos.
- **Plegables:** Las APIs de Window Manager (Android) para detectar el ángulo de bisagra y cambiar el layout en milisegundos son nativas. Implementarlas en un framework externo suele depender de que alguien más escriba el "plugin", lo que te quita control sobre la innovación.

¿Qué es un aplicativo nativo y cómo se clasifican?

Como historiador, debo decirte que el término "nativo" ha evolucionado. Un aplicativo nativo es aquel que ha sido desarrollado específicamente para una plataforma (SO), utilizando las herramientas y lenguajes proporcionados por el fabricante del sistema (Apple o Google).

Se clasifican de la siguiente manera:

1. **Nativo Puro (OEM):**
   - Usa los lenguajes oficiales: **Swift** para iOS, **Kotlin/Java** para Android.
   - Usa los kits de desarrollo oficiales (iOS SDK / Android SDK).
   - **Ejemplo:** La app de Ajustes de tu teléfono.
2. **Nativo Compilado (AOT - Ahead of Time):**
   - Se escribe en un lenguaje distinto (como **Dart** en Flutter), pero se **compila a código de máquina** (binario) antes de ejecutarse.

- No usa los componentes de UI del sistema, pero el procesador lo lee como si fuera nativo.

3. **Nativo con Lógica Compartida:**
   - La interfaz es nativa pura, pero la lógica compleja reside en una librería compartida en **C++, Rust o Kotlin Multiplatform (KMP)**.
   - Es el "híbrido de los ingenieros": lo mejor de los dos mundos.
   - **Ejemplo:** Netflix, Duolingo, o la app de búsqueda de Google.
4. **Híbrido Nativo (Interpretado):**
   - Aplicaciones como **React Native**. El código se interpreta en tiempo de ejecución, pero invoca componentes nativos reales. Se le llama "nativo" porque el botón que ves es un botón del sistema, aunque la orden de crearlo venga de JavaScript.